

LAB ACTION PLAN FOR WEEK 7

Objective:

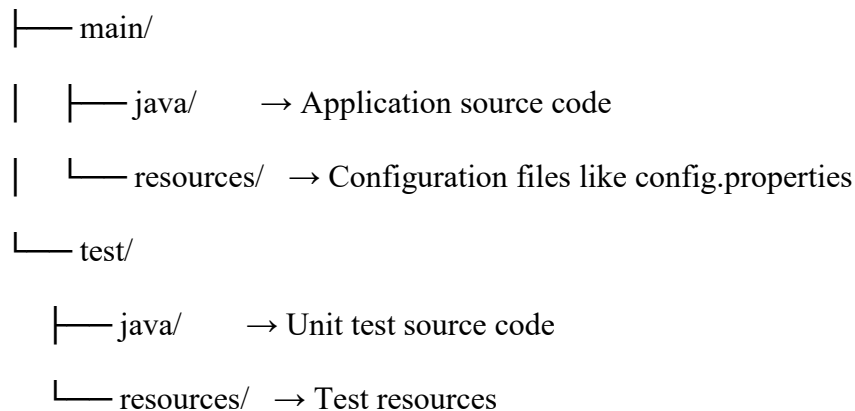
To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications.
src/



The compiled files and reports are generated in the **target/** directory after a successful build.

2. Steps to Perform Maven Build and Testing

----mvn clean install

- **clean:** Deletes the previous build (**target/** folder)
- **install:** Builds, tests, and installs the package to local **.m2** repository

After execution:

- **target/** folder contains compiled **.class** files and the final **.jar** or **.war**

- Artifact is stored in:
`~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In `pom.xml`:

```
<dependency>  
  
<groupId>com.google.code.gson</groupId>  
  
<artifactId>gson</artifactId>  
  
<version>2.10</version>  
  
</dependency>
```

After running:

```
mvn clean install
```

Check:

- Gson JAR is downloaded to `.m2/repository/com/google/code/gson/gson/`
- If version is wrong or not found → **BUILD FAILURE** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└─ 1.1. Launch Eclipse workspace

Step 2. Install Maven Plugin (if not installed)

└─ 2.1. Go to "Help" in the top menu

└─ 2.1.1. Click "Eclipse Marketplace"

└─ 2.1.2. Search for "Maven Integration for Eclipse"

└─ 2.1.3. Install the plugin if not already installed

Step 3. Create a New Maven Project

└─ 3.1. File -> New -> Project...

└─ 3.1.1. Expand "Maven"

- └─ 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

- └─ 4.1. Select workspace location (default or custom)
- └─ 4.2. Click "Next"

Step 5. Choose Maven Archetype

- └─ 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")
- └─ 5.2. Click "Next"

Step 6. Define Project Metadata

- └─ 6.1. Group ID: (e.g., com.example)
- └─ 6.2. Artifact ID: (e.g., my-maven-project)
- └─ 6.3. Version: (default is usually fine)
- └─ 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.

Step 7. Maven Project Created

- └─ 7.1. Project structure is generated with a standard Maven layout
- └─ 7.2. Includes:
 - └─ src/main/java (for Java source code)
 - └─ src/test/java (for test code)
 - └─ pom.xml (Maven configuration file)

Step 8. Update Project Settings (if needed)

- └─ 8.1. Right-click on the project -> Maven -> Update Project...
- └─ 8.2. Ensure dependencies are up to date

Step 9. Build and Run Maven Project

- └─ 9.1. Right-click on App.java -> Run As -> Maven Clean
 - └─ 9.1.1. Right-click on App.java -> Run As -> Maven Install

- └─ 9.1.2. Right-click on App.java -> Run As -> Maven Test

- └─ 9.1.3. Right-click on App.java -> Run As -> Maven Build

Step 10. In the Maven Build dialog:

- └─ Enter Goals: clean install test

- └─ Click on Apply -> Click on Run

Step 11. Check console for BUILD SUCCESS message.

Step 12. Run the application:

- └─ Right-click on App.java -> Run As -> Java Application

- └─ Output: "Hello World" displayed.

Creation of Maven web Java Project

Step 1: Open Eclipse

- └─ 1.1 Launch Eclipse IDE.

- └─ 1.2 Select or create a workspace.

Step 2: Create a New Maven Project

- └─ 2.1. File -> New -> Project...

- └─ 2.1.1. Expand "Maven"

- └─ 2.1.2. Select "Maven Project" and click "Next"

Step 3: Choose Maven Archetype

- └─ 3.1. Select an archetype(e.g "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

- └─ 3.2. Click "Next"

Step 4: Configure the Maven Project

- └─ 4.1 Group Id: Enter a group ID (e.g., com.example).

- └─ 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

- └─ 4.3 Click ****Finish**** to create the project.

Step 5: Add Maven Dependencies

└─ 5.1 Open the ****pom.xml**** file in the Maven project.

└─ 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

└─ Example:

```
``xml

<dependency>

<groupId>javax.servlet</groupId>

<artifactId>javax.servlet-api</artifactId>

<version>4.0.1</version>

<scope>provided</scope>

</dependency>

...
```

Step 6:- Configure server:

└─ Window -> Show View -> Servers

└─ Add server -> Select Tomcat v9.0 server -> Click Next

└─ Configure server options (e.g., ports, server location).

Step 7:- Modify 'tomcat-users.xml':

└─ Add role and user details under <tomcat-users> tag.

Step 8:- Build the project:

└─ Right-click on index.jsp -> Run As -> Maven Clean

└─ Right-click on index.jsp -> Run As -> Maven Install

└─ Right-click on index.jsp -> Run As -> Maven Test

└─ Right-click on index.jsp -> Run As -> Maven Build

Step 9. In the Maven Build dialog:

- └ Enter Goals: clean install test
- └ Click on Apply -> Click on Run

Step 10. Check console for BUILD SUCCESS message.

Step 11. Run the application:

- └ Right-click on index.jsp -> Run As -> Run on Server
- └ Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github

3. Configure Java Version via Compiler Plugin

In `pom.xml`:

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.11.0</version>
<configuration>
<source>17</source>
<target>17</target>
</configuration></plugin>
```

Check:

- Run `mvn package`
- Inspect generated JAR: `jar tf target/myapp.jar`

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```

public class AppTest {

    @Test

    public void testSum() {

        assertEquals(5, 2 + 3);

    }

}

```

Run:

```
mvn test
```

Check:

- target/test-classes/ → compiled test .class files
 - target/surefire-reports/ → .txt or .xml test results
- If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause BUILD FAILURE
- Maven shows precise error in console
- Fix the dependency tag → re-run `mvn clean install`

6. Adding Resource Files

Put config.properties inside:

```
src/main/resources/config.properties
```

To read:

```

InputStream input =
getClass().getClassLoader().getResourceAsStream("config.properties");

```

After build, check target/classes/ → file should exist there.

7. Multi-Module and Parent Projects

Structure:

```

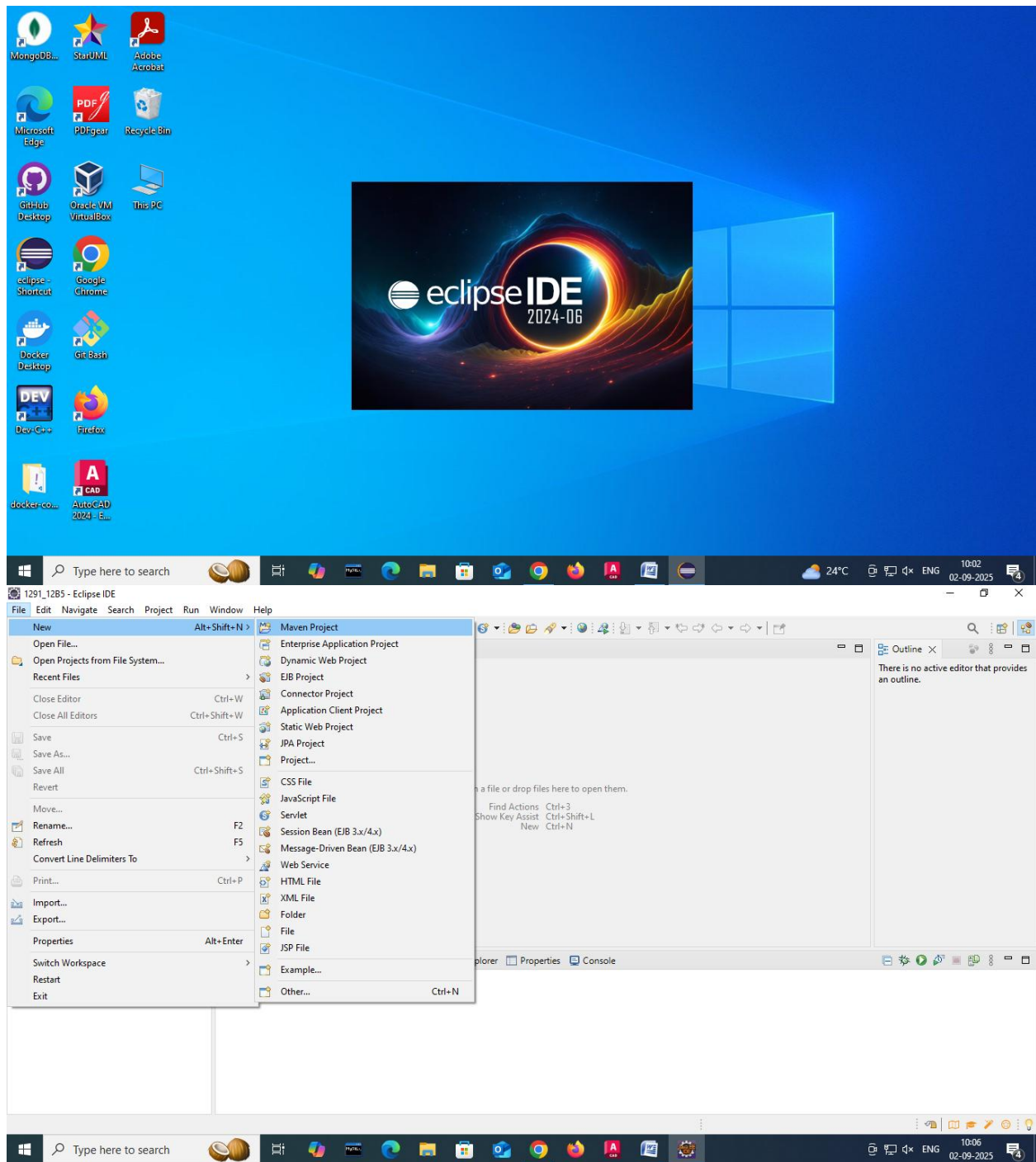
parent/
├── pom.xml (packaging: pom)
├── core/
│   └── pom.xml
└── web/
    └── pom.xml

```

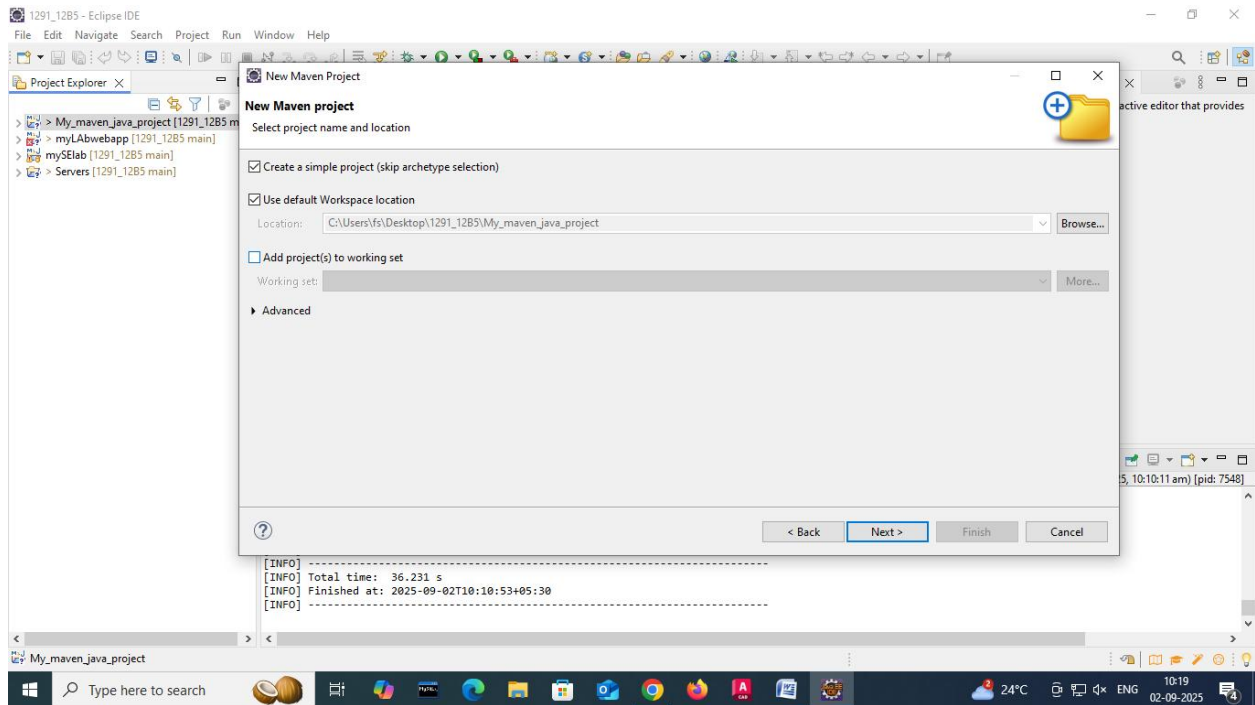
- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

Creating a Multi-Module Maven Project:

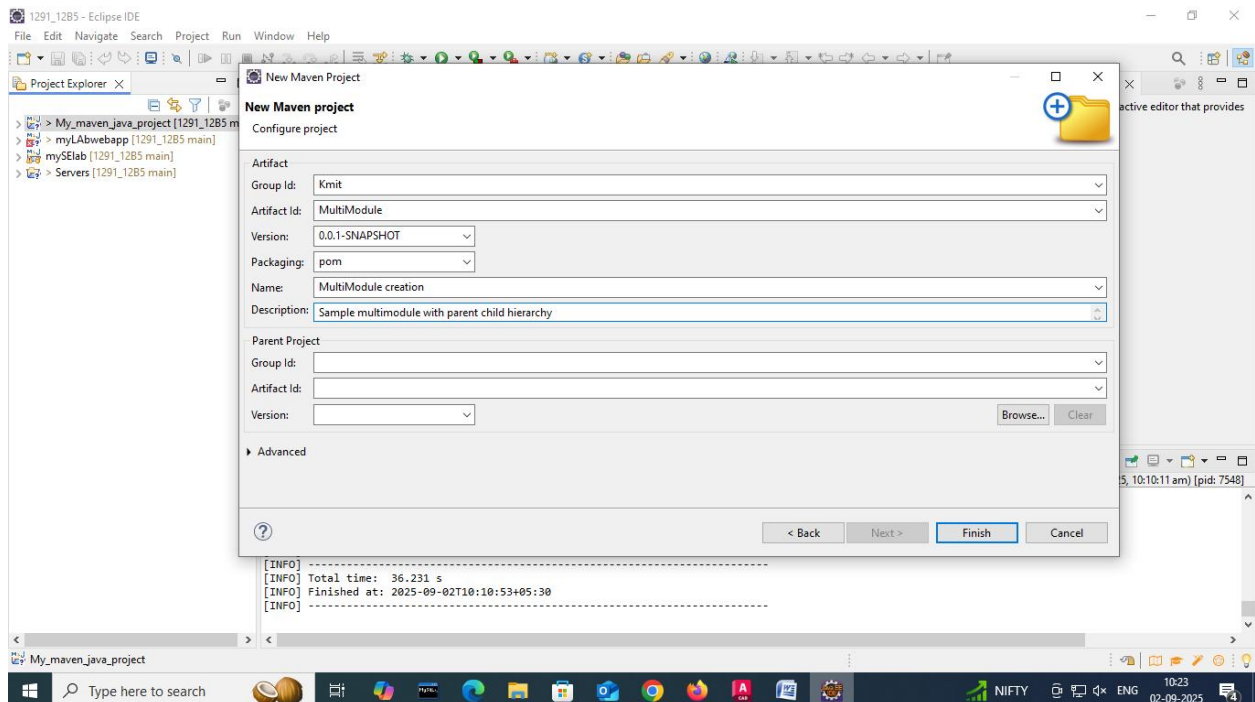
1. Open eclipse-> file-> new -> other -> Maven -> MavenProject ->click on next.



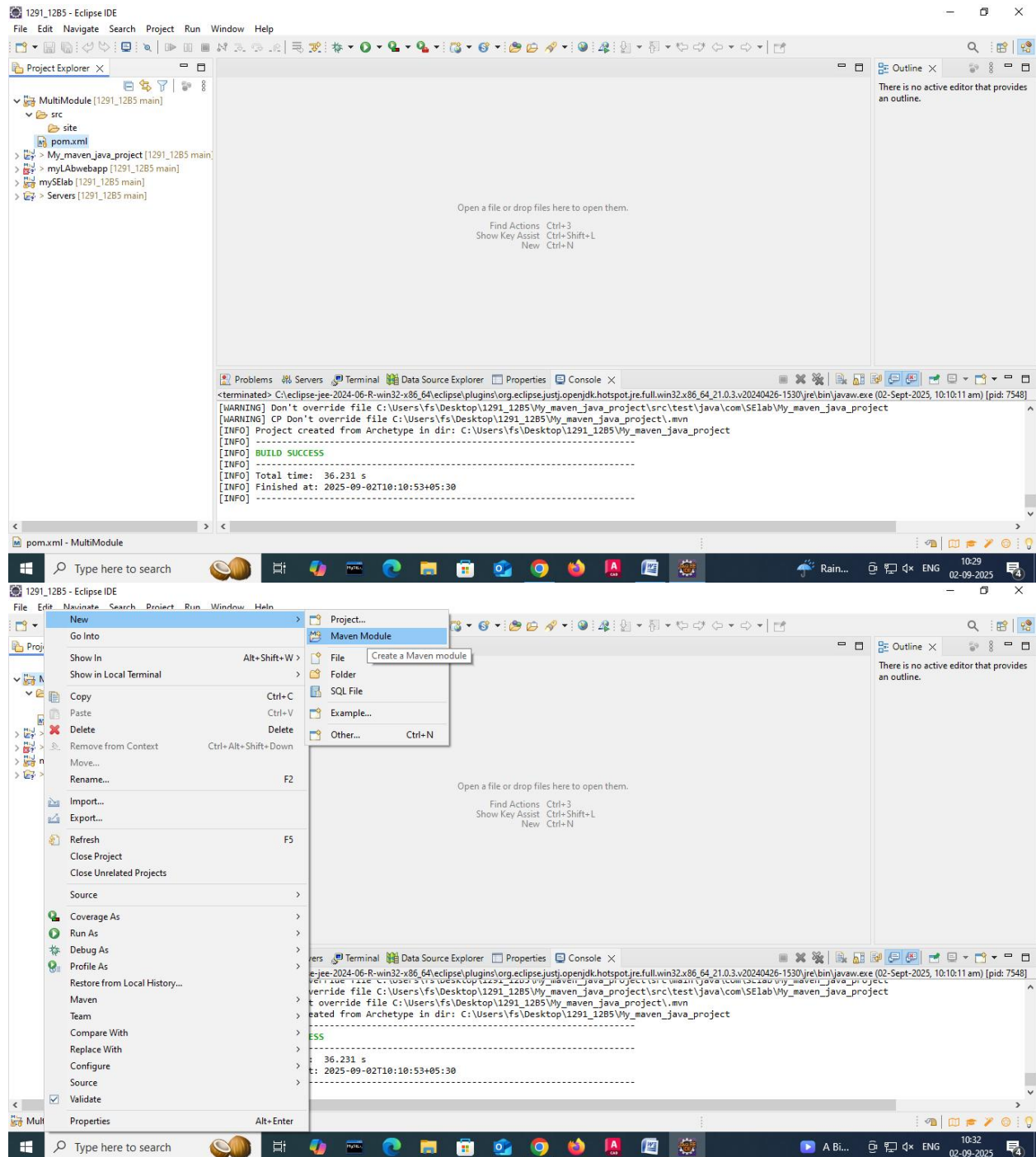
2. Check the first option: Create a Simple project(skip archetype selection) and click on next.



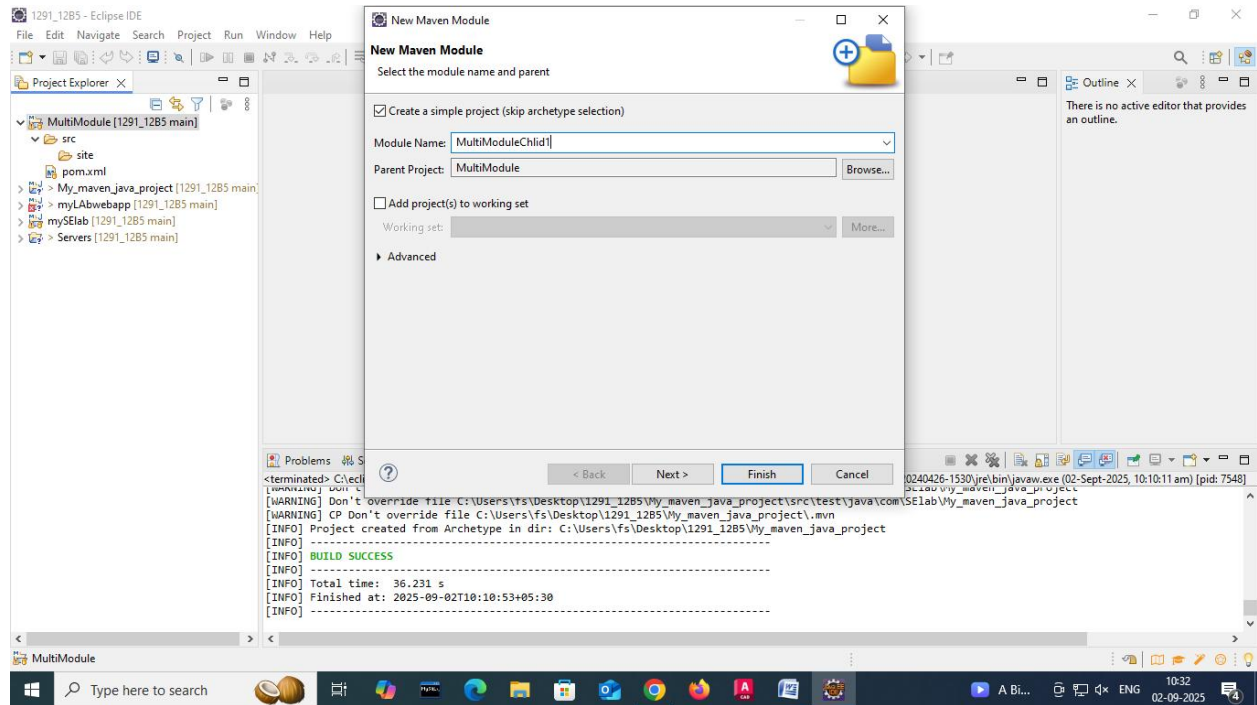
3. Give groupId: KMIT, artifact ID: MultiModule, in the packaging tab select the drop down and opt for pom.
4. Give Name: Multimodule creation and Description: Sample multimodule with parent child hierarchy and click on next.



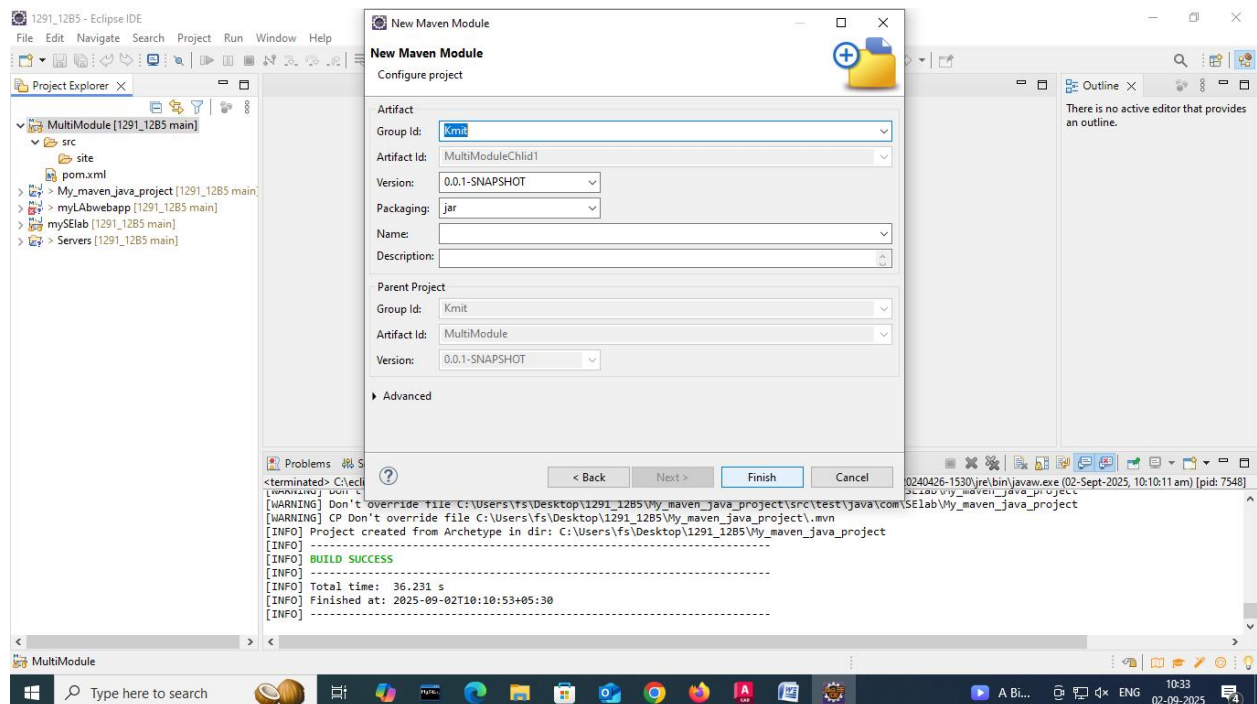
- Now the Multimodule folder is created and displayed in the project explorer on the left. Right click on the folder and opt new -> mavenModule

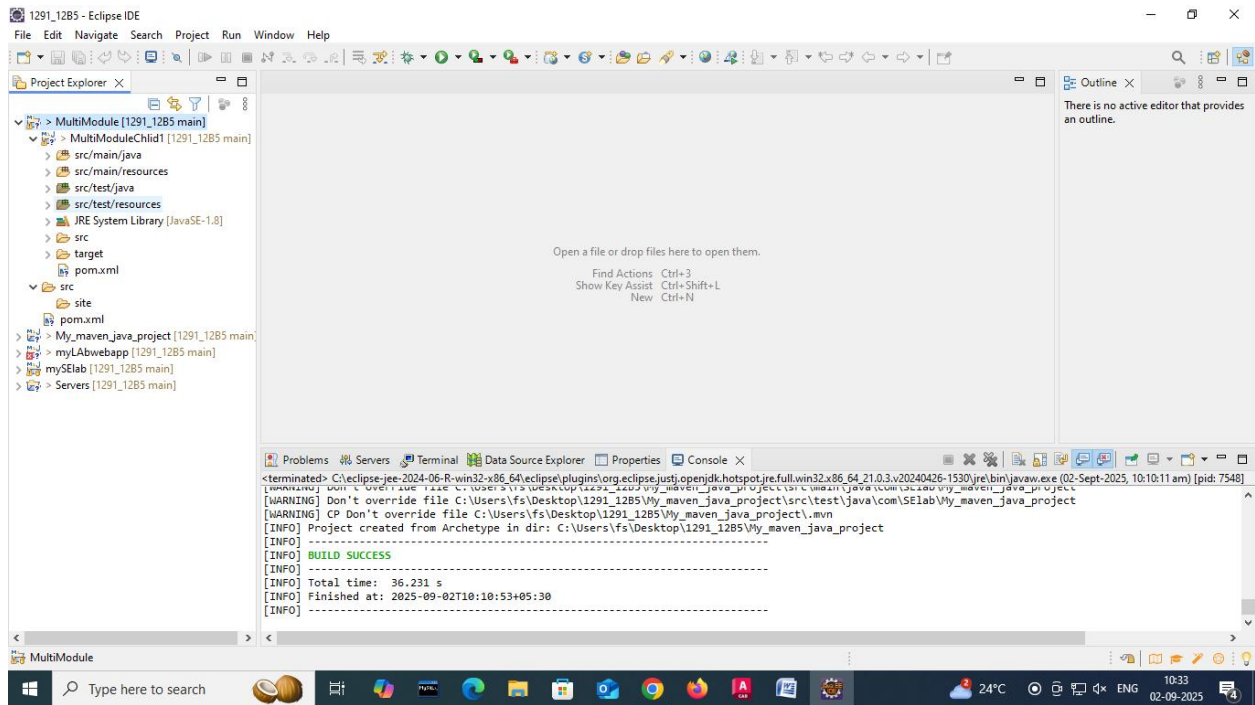


6. Check the option Create a Simple project(skip archetype selection, give the artifactID: MultiModuleChild1 and click on next and finish.

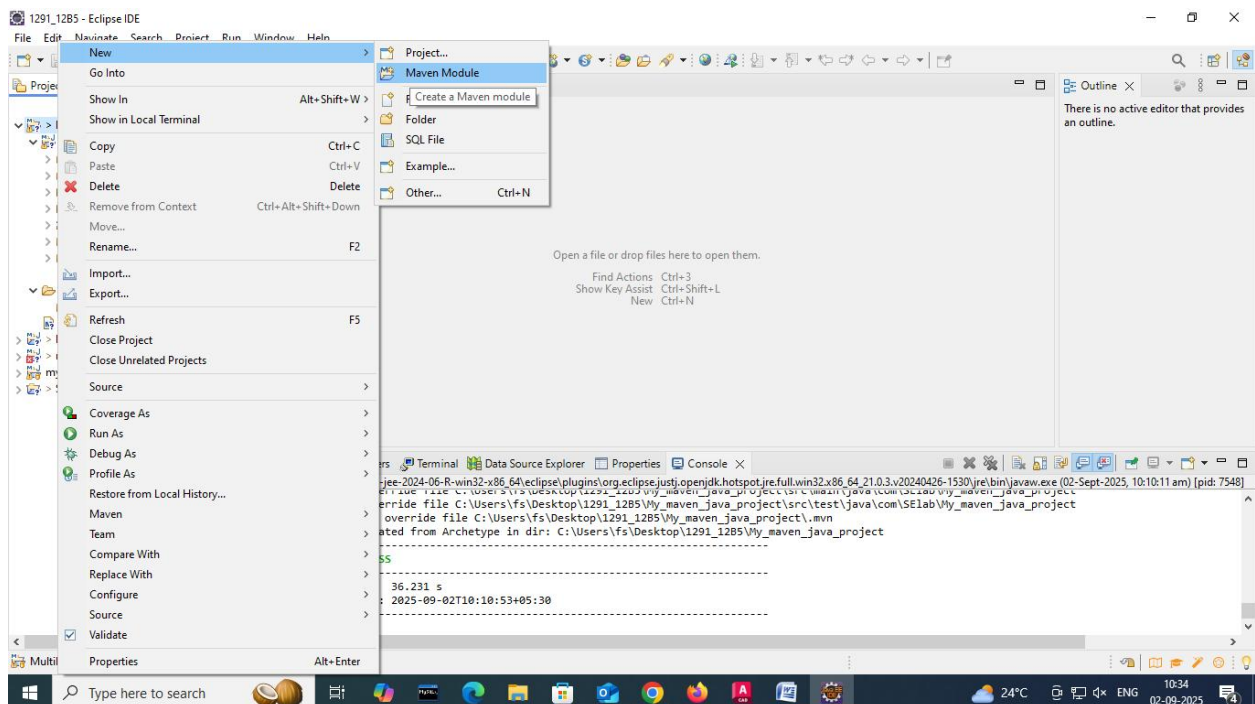


7. Now you can see, the module1 we created is displayed in the project explorer with a separate pom.xml file.

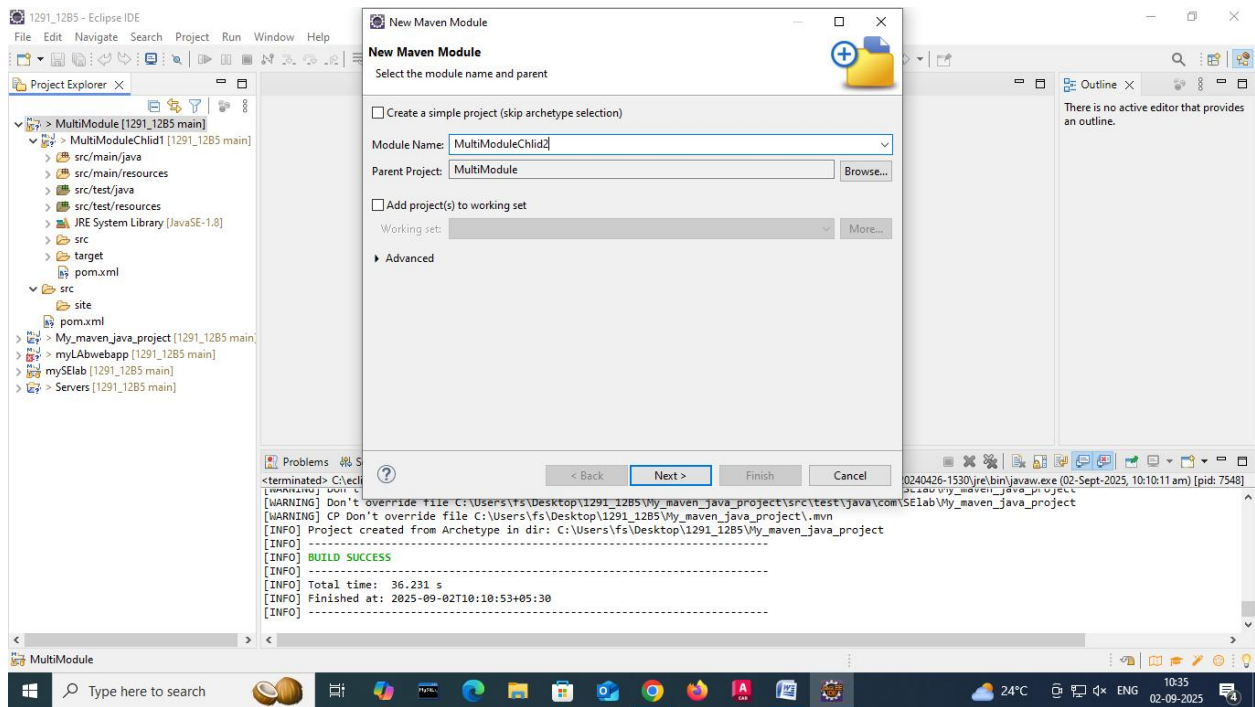




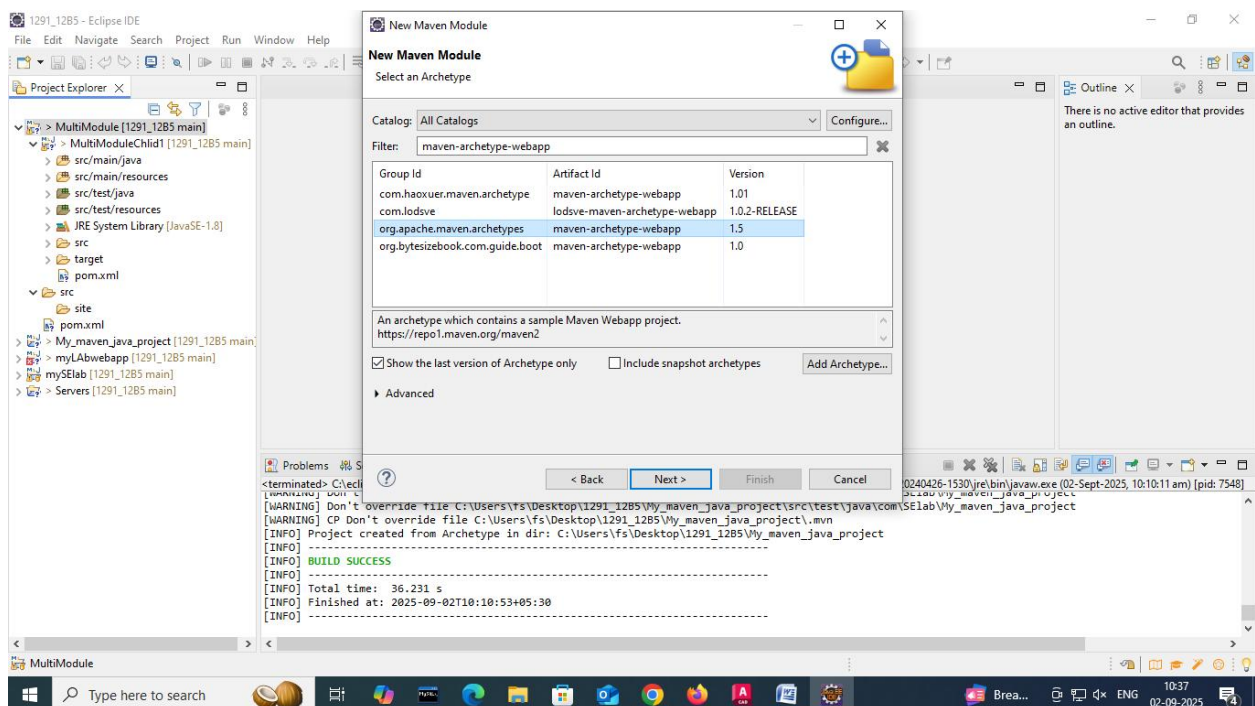
8. . Right click on the MultiModule folder in the project explorer and opt new -> mavenModule



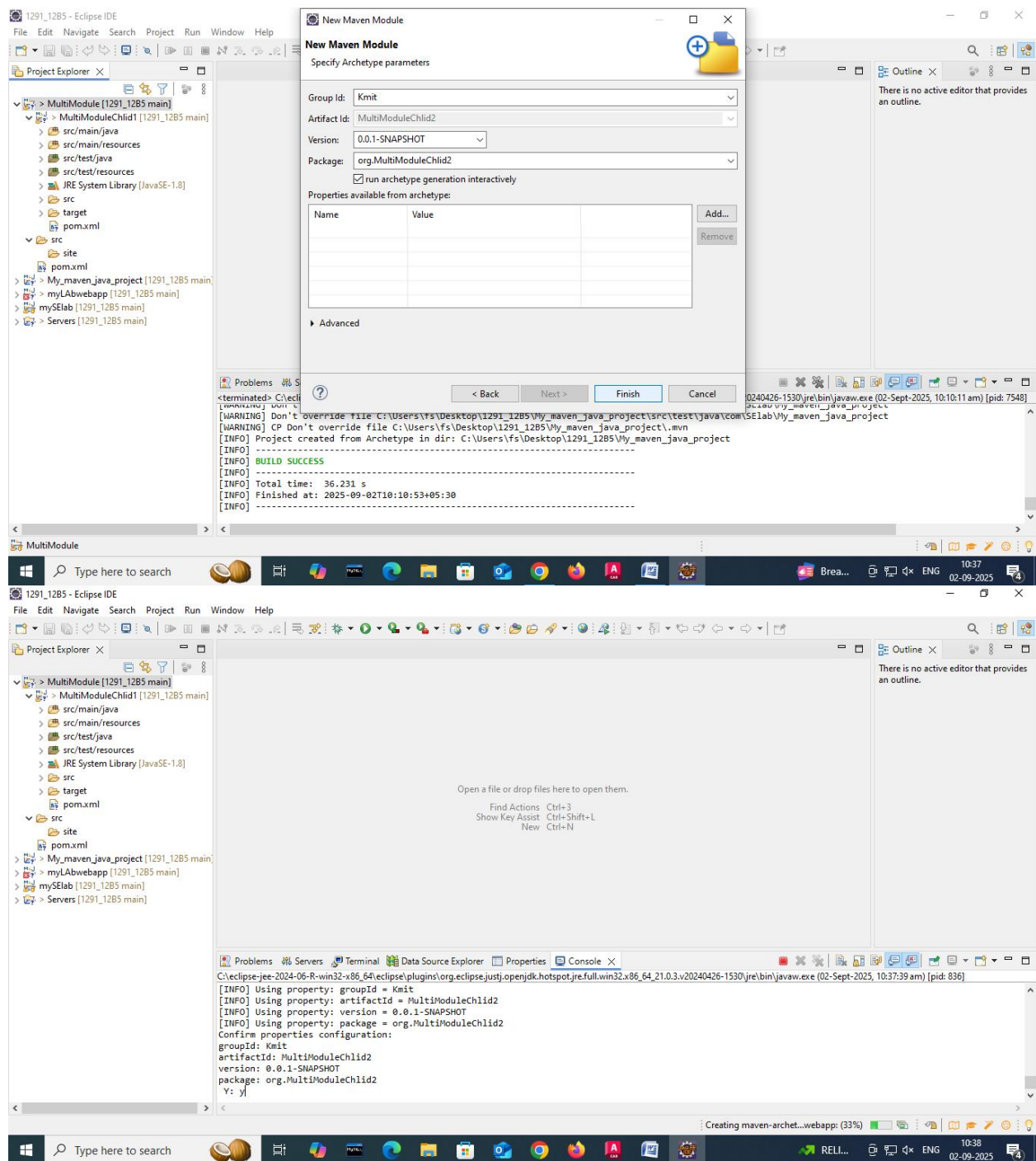
9. Give the artifactID: MultiModuleChild2 and click on next.



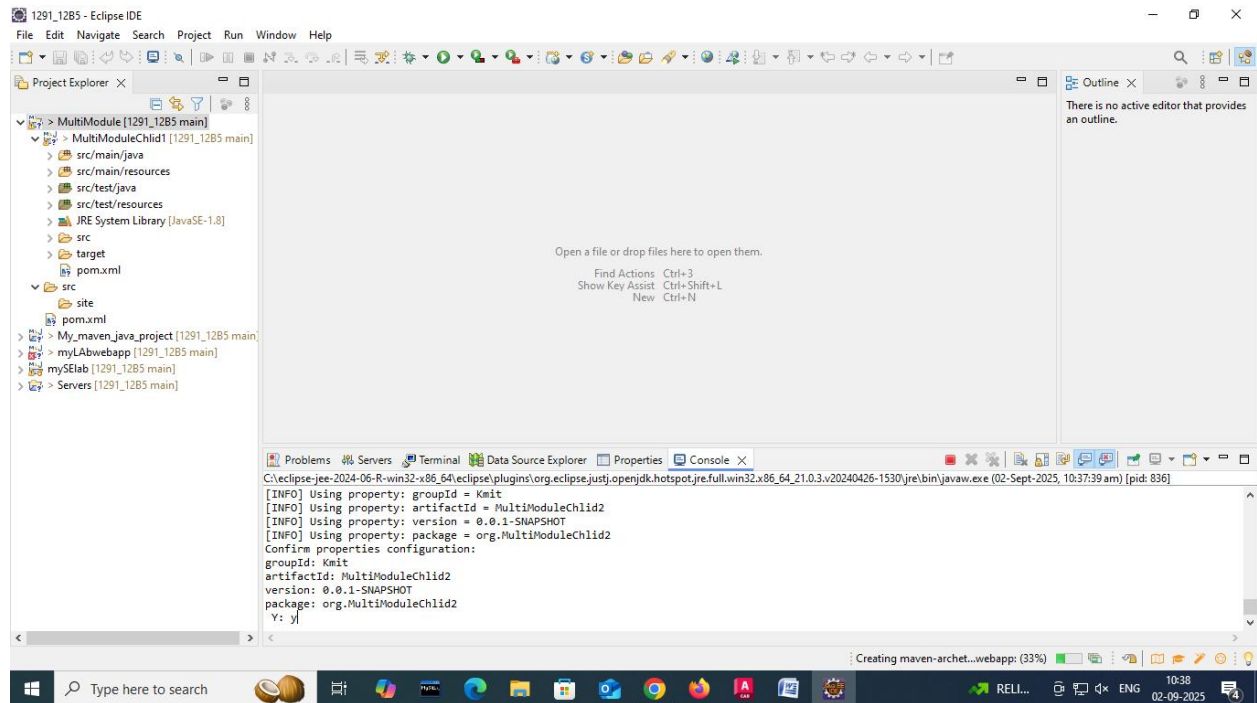
10. In filters search for maven-archetype-webapp, select the version 1.1/1.5 and click on next.



11. Now click on finish, and confirm with a Y in the console.



12. Now you can watch the new child is been added to the parent project.



Note: Childs or Modules can be added to the projects who have the packaging type as pom. If we try to create a module within child1 it is not possible since its packaging is opted to JAR.

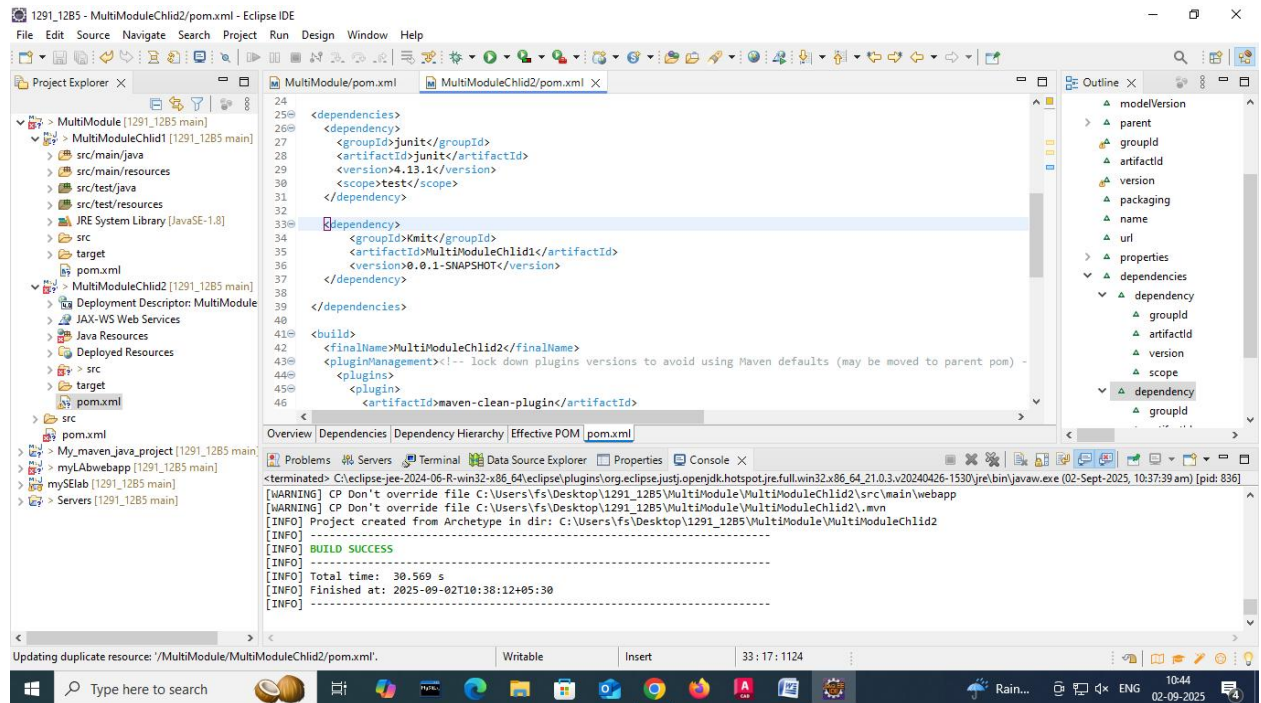
Linking two modules in a Parent Project:

Go to the module u want to connect. And add a dependency to other module on which the current module depends. For example if my child2 is dependent on child1. I have to change the dependencies in the pom.xml by adding a new dependency of child1.

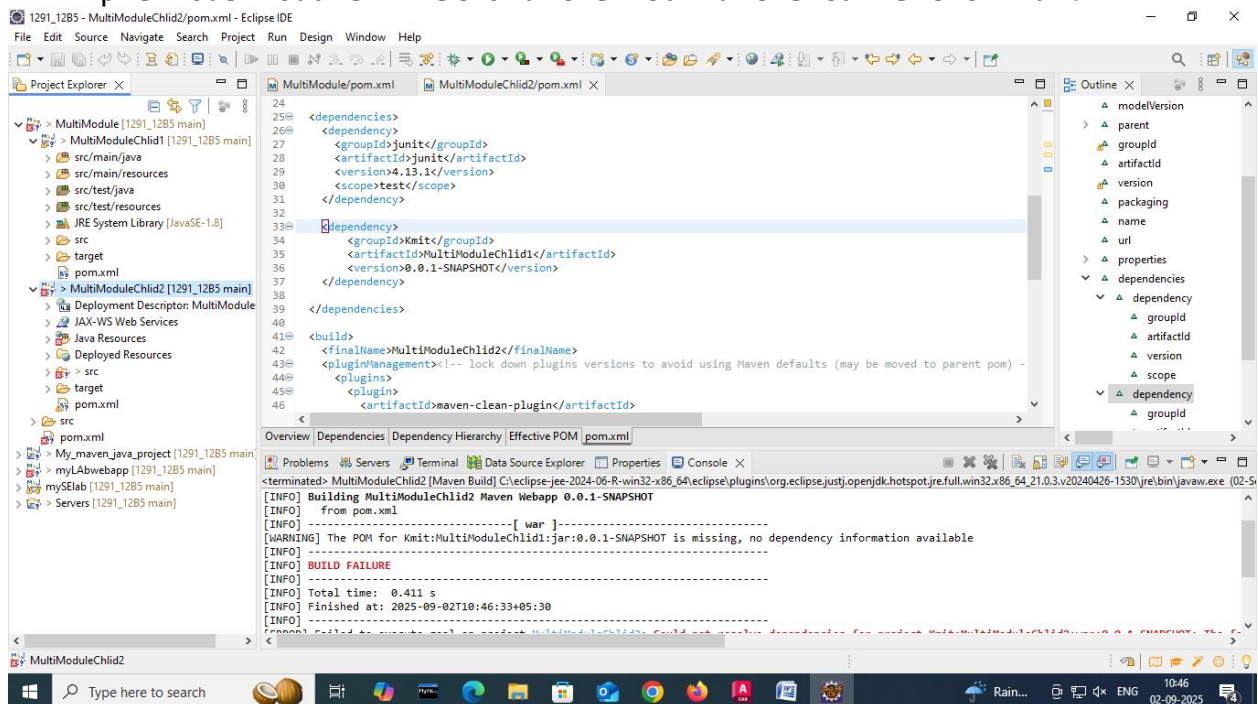
Steps to add dependency:

1. Goto pom.xml of child2.
2. Navigate to dependencies tag. Update the dependencies with child1 dependency as given below, and save pom.xml after update.

```
<dependency>
  <groupId>KMIT</groupId>
  <artifactId>MultimoduleChild1</artifactId>
  <version>0.0.1-SNAPSHOT</version>
</dependency>
```



- Now build the child2 by right clicking it and opting run as -> maven build.
- The build will be a *failure*. Because we haven't build the parent project and the child project. Child1 is dependent on parent. And Child2 is dependent on child2. SO we have to build ass the previous module first and then build the current child2.



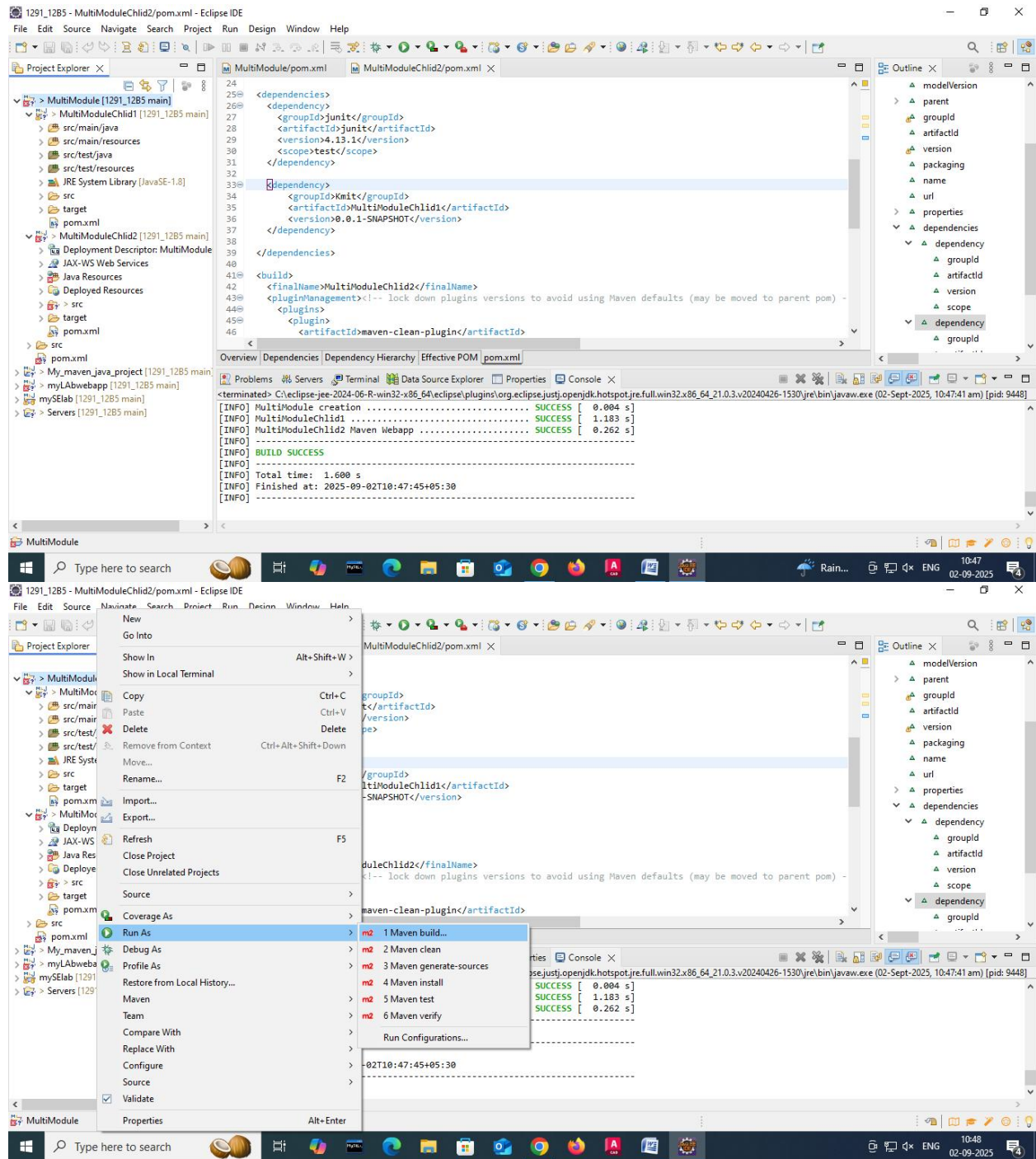
5. After building parent and child1, build child2 now the build will be a *Success*

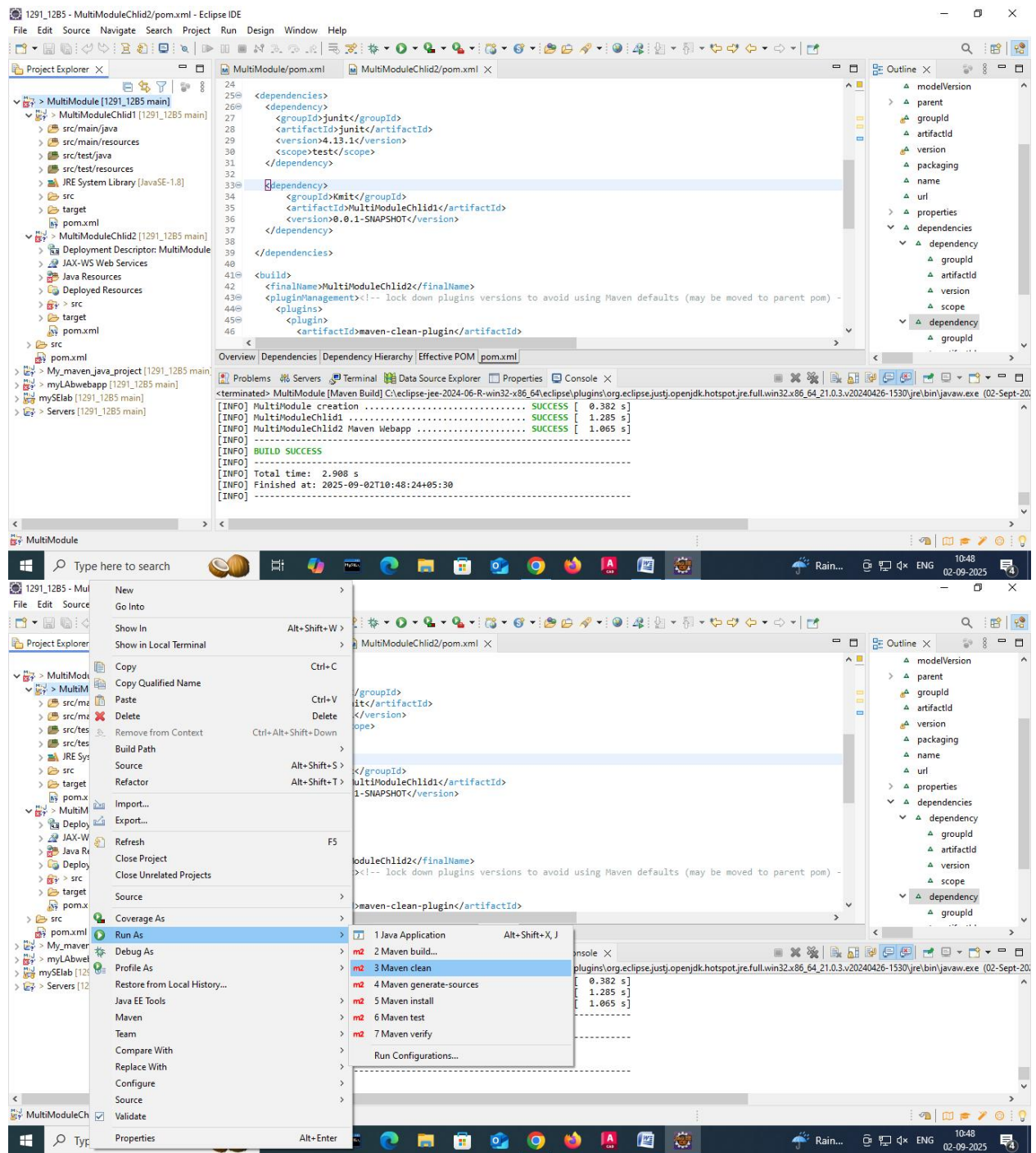
The screenshot shows the Eclipse IDE interface with the following components:

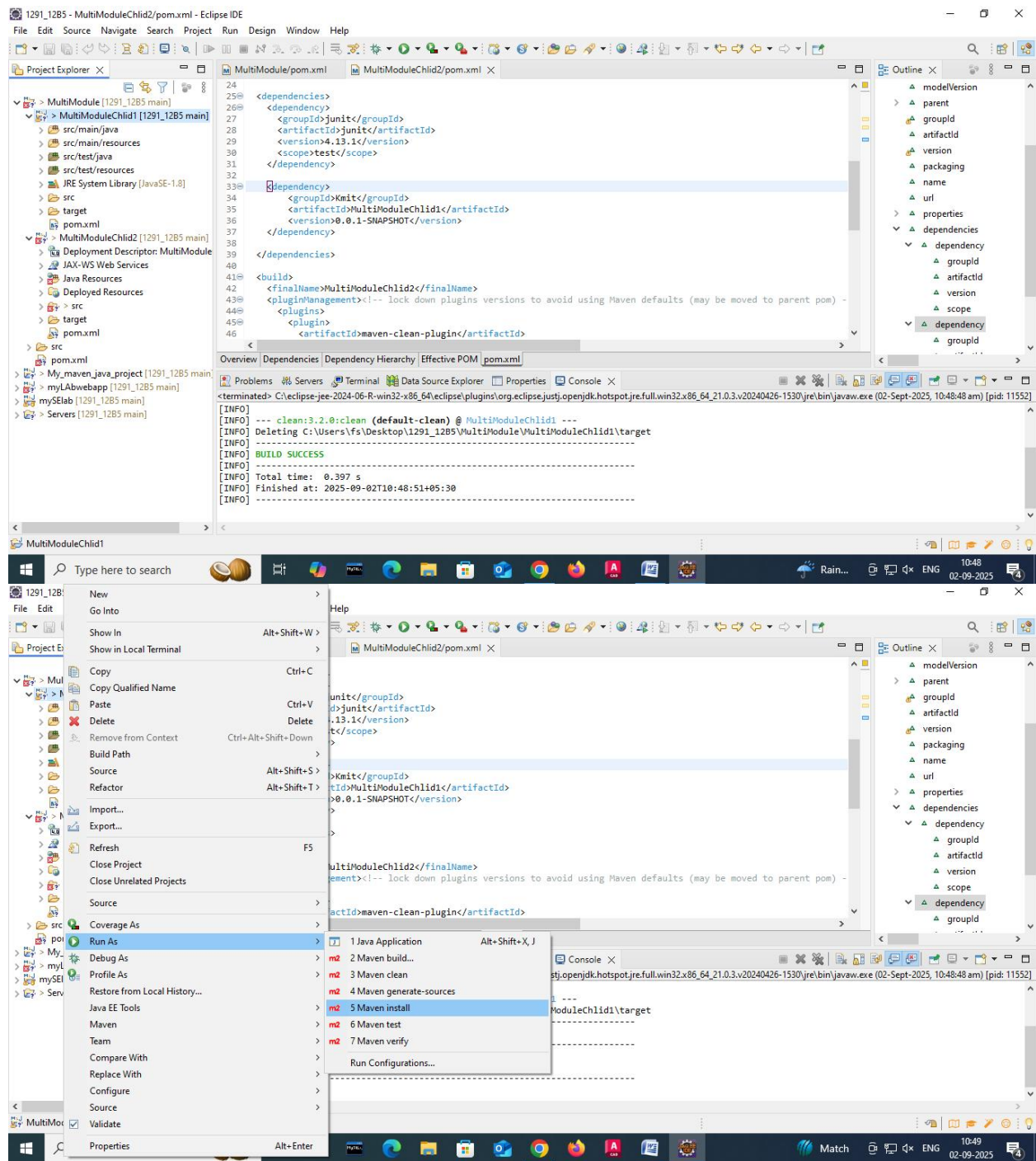
- Project Explorer:** Displays the project structure for 'MultiModule' and its children 'MultiModuleChild1' and 'MultiModuleChild2'.
- MultiModule/pom.xml:** The main POM file, showing dependencies on 'junit' and 'MultiModuleChild1', and a build section with 'maven-clean-plugin'.
- MultiModuleChild2/pom.xml:** The child POM file, showing a dependency on 'MultiModuleChild1'.
- Console:** Displays the build output, including the following log entries:

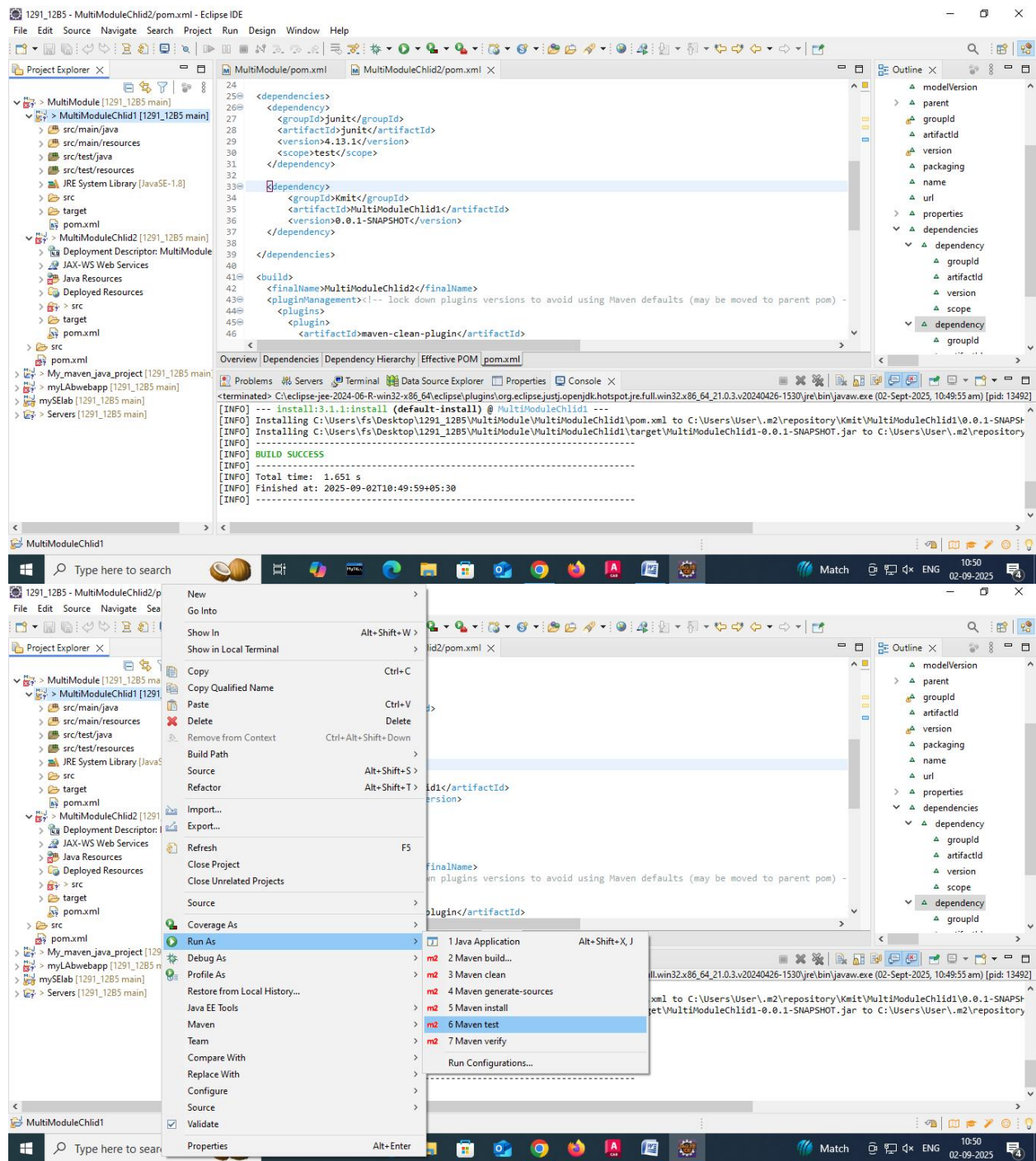
```
<terminated> C:\eclipse-jee-2024-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.3.v20240426-1530\jre\bin\javaw.exe (02-Sept-2025, 10:46:50 am) [pid: 8452]
[INFO] MultiModule creation ..... SUCCESS [ 7.071 s]
[INFO] MultiModuleChild1 ..... SUCCESS [ 0.013 s]
[INFO] MultiModuleChild2 Maven Webapp ..... SUCCESS [ 0.001 s]
[INFO] BUILD SUCCESS
[INFO] Total time: 7.321 s
[INFO] Finished at: 2025-09-02T10:46:59+05:30
[INFO]
```
- Outline:** Shows the project structure and dependencies.

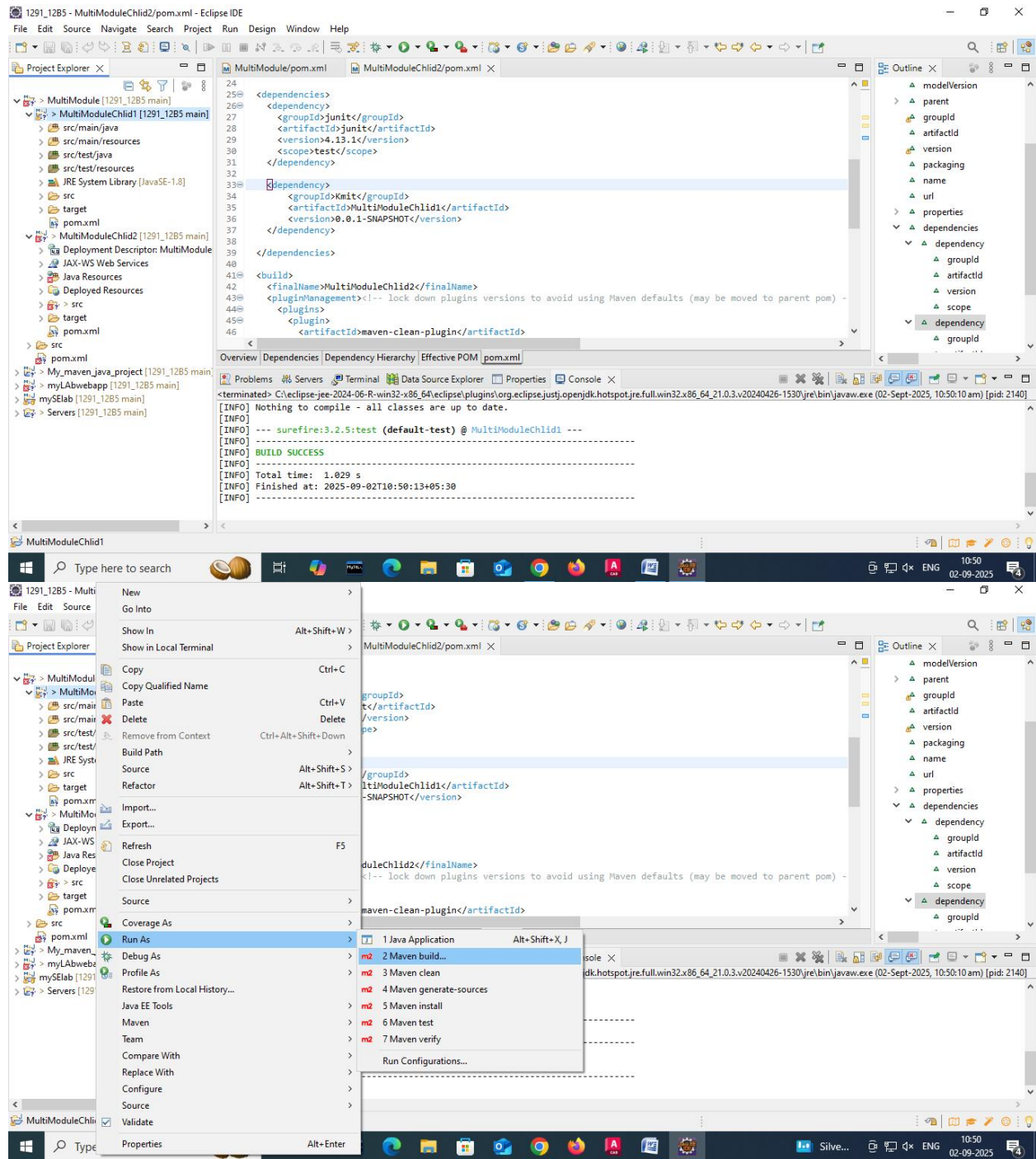
The bottom of the image shows the Windows taskbar with the system clock at 10:47 on 02-09-2025.











1291_1285 - MultiModuleChild2/pom.xml - Eclipse IDE

File Edit Source Navigate Search Project Run Design Window Help

Project Explorer

- MultiModule [1291_1285 main]
 - MultiModuleChild1 [1291_1285 main]
 - src/main/java
 - src/main/resources
 - src/test/java
 - src/test/resources
 - JRE System Library [JavaSE-1.8]
 - src
 - target
 - pom.xml
 - MultiModuleChild2 [1291_1285 main]
 - Deployment Descriptor: MultiModule
 - JAX-WS Web Services
 - Java Resources
 - Deployed Resources
 - src
 - target
 - pom.xml
 - src
 - pom.xml
- My_maven_java_project [1291_1285 main]
- myLabwebapp [1291_1285 main]
- mySElab [1291_1285 main]
- Servers [1291_1285 main]

MultiModuleChild1

MultiModule/pom.xml

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>Kmit</groupId>
    <artifactId>MultiModuleChild1</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
</dependencies>

<build>
  <finalName>MultiModuleChild2</finalName>
  <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
    <plugins>
      <plugin>
        <artifactId>maven-clean-plugin</artifactId>
```

Outline

- modelVersion
- parent
- groupId
- artifactId
- version
- packaging
- name
- url
- properties
- dependencies
 - dependency
 - groupId
 - artifactId
 - version
 - scope
 - dependency
 - groupId

bin/javaw.exe (02-Sept-2025, 10:50:10 am) [pid: 2140]

MultiModuleChild1

MultiModule/pom.xml

Overview | Dependencies | Dependency Hierarchy | Effective POM | pom.xml

Problems Servers Terminal Data Source Explorer Properties Console

<terminated> MultiModuleChild1 [Maven Build] C:\eclipse-jee-2024-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.3.v20240426-1530\jre\bin/javaw.exe (02-Sept-2025, 10:50:10 am) [pid: 2140]

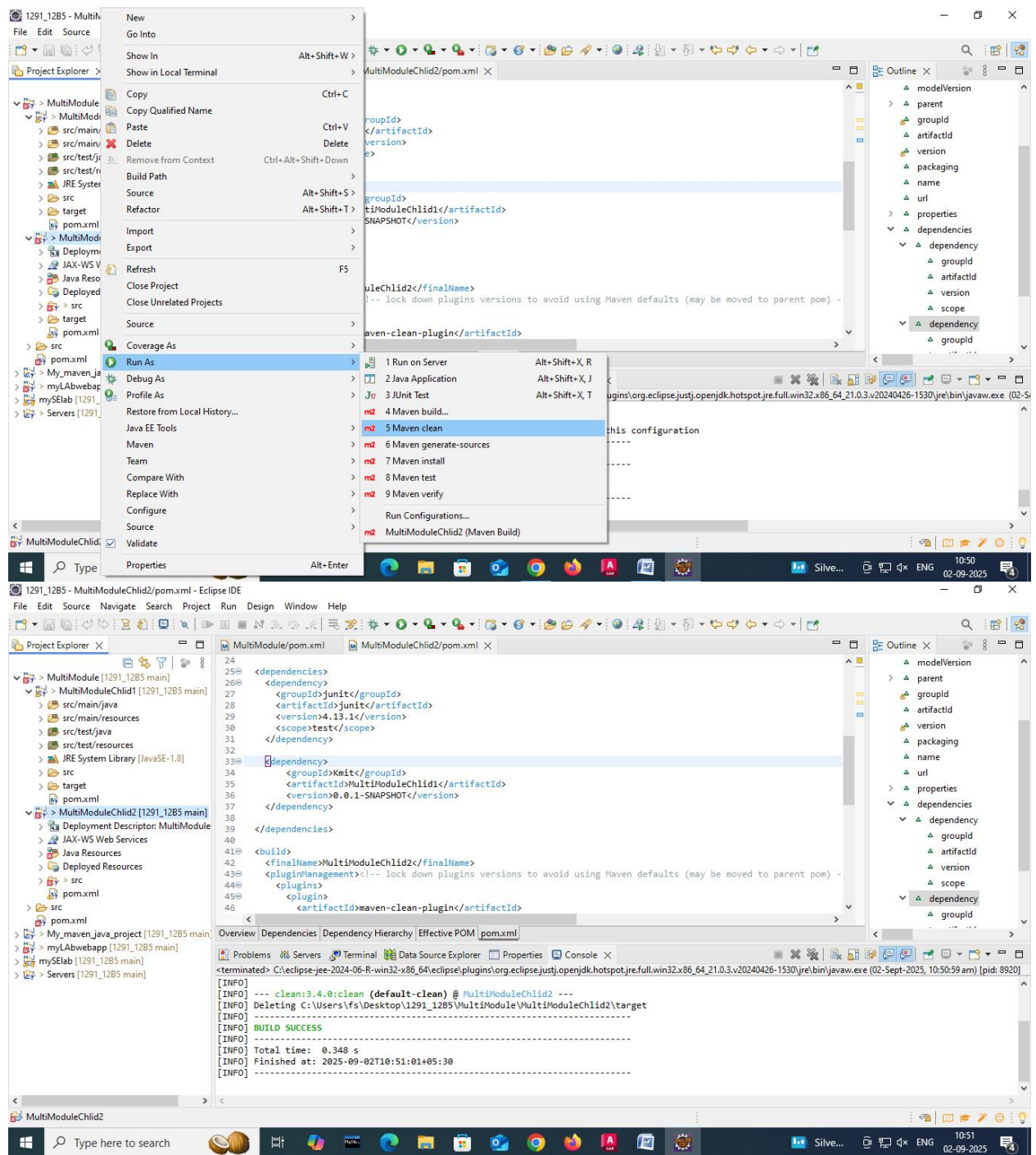
[INFO] --- surefire:3.2.5:test (default-test) @ MultiModuleChild1 ---

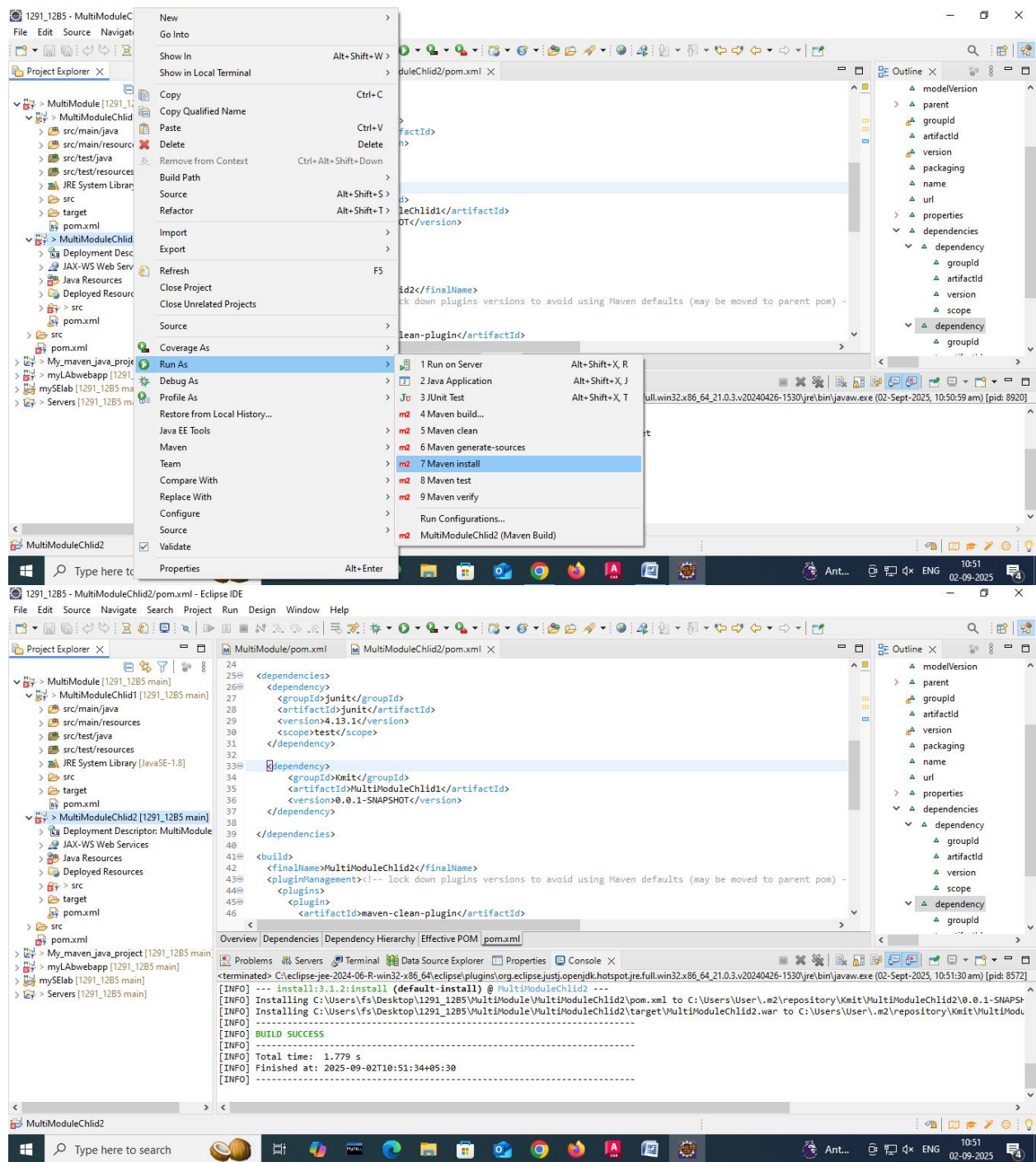
[INFO] Skipping execution of surefire because it has already been run for this configuration

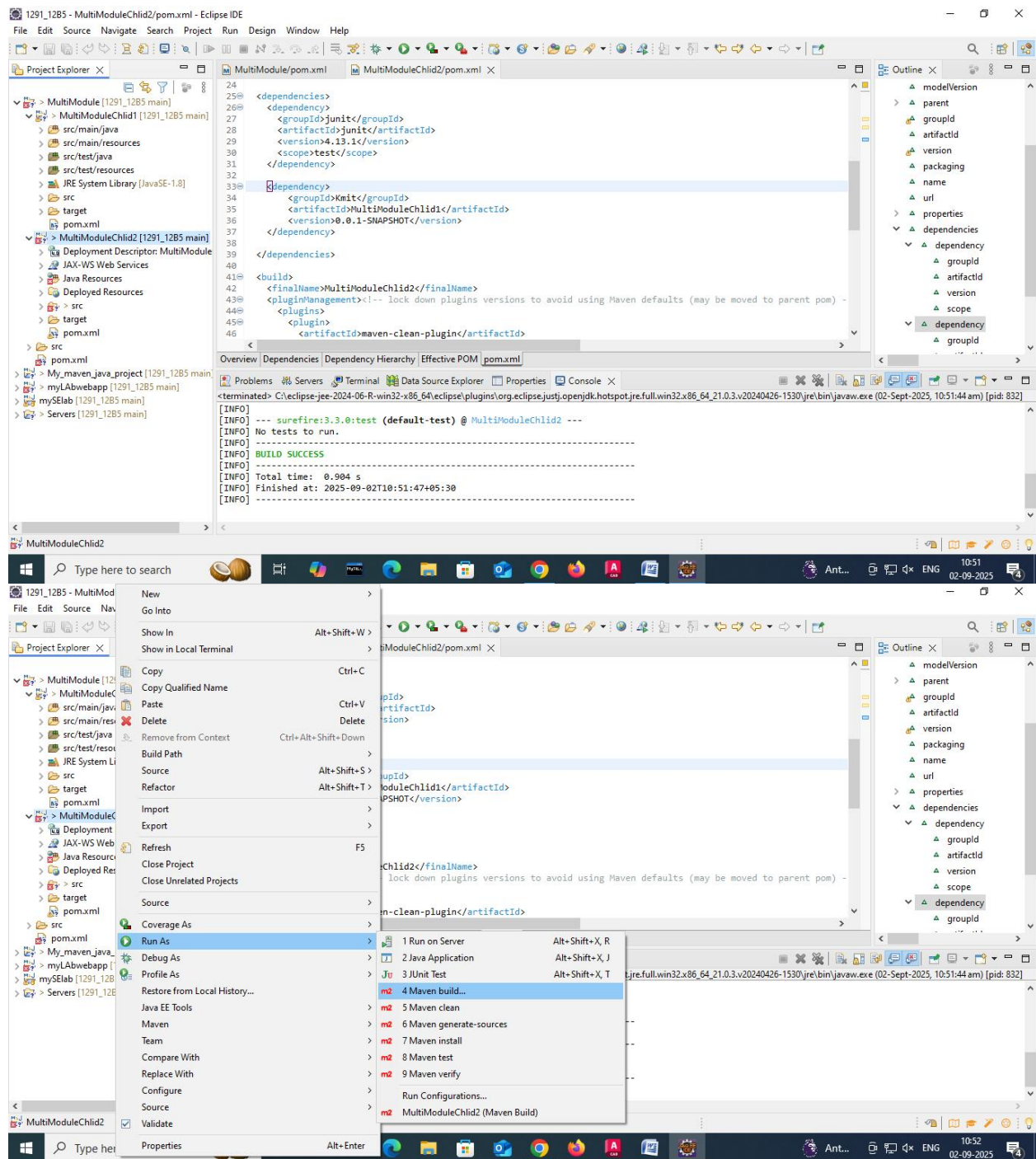
[INFO] BUILD SUCCESS

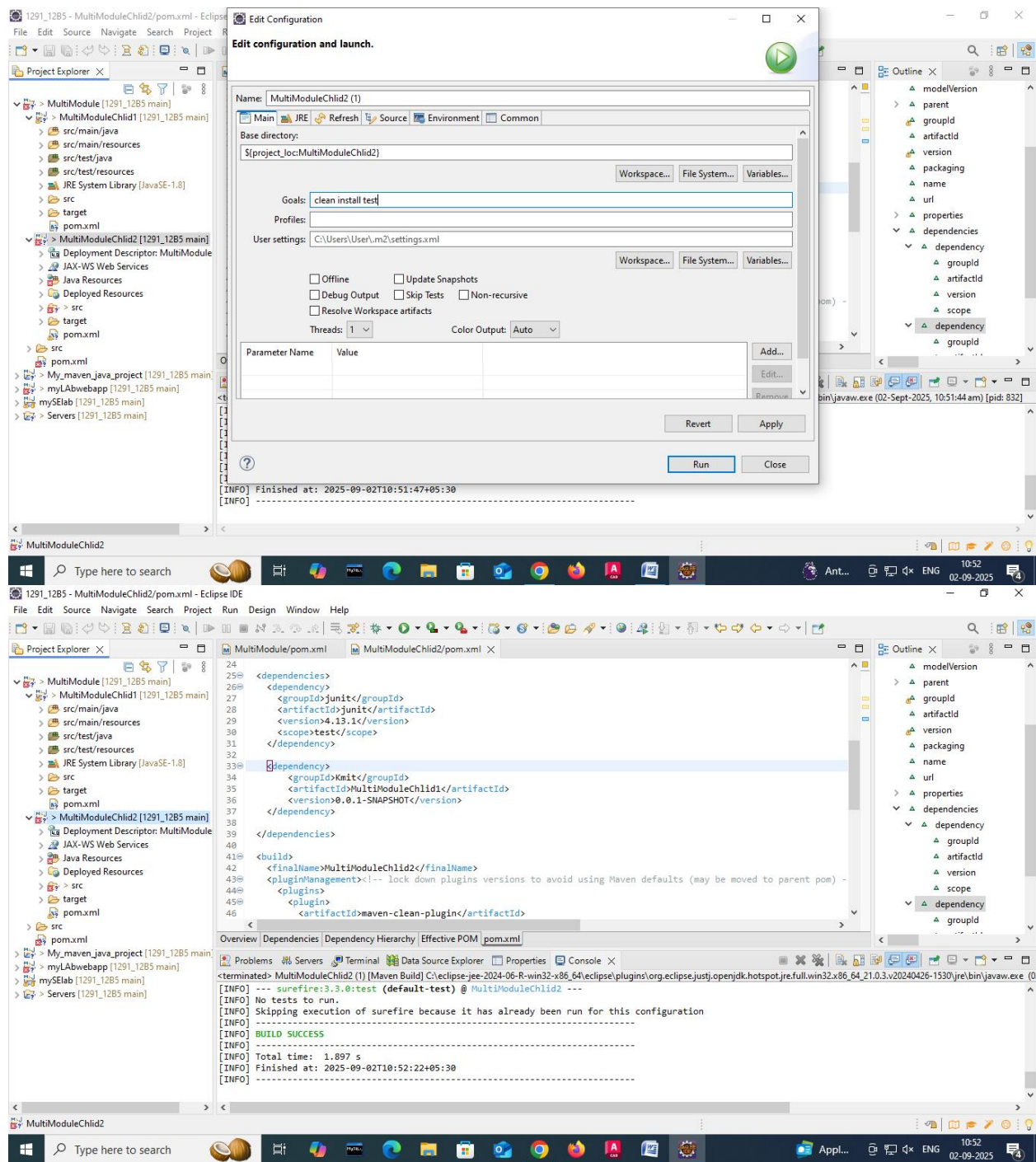
[INFO] Total time: 1.716 s

[INFO] Finished at: 2025-09-02T10:50:46+05:30









8. Executable JARs

Add to pom.xml:

```
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<configuration>
<archive>
<manifest>
<mainClass>com.example.Main</mainClass>
</manifest>
</archive>
</configuration>
</plugin>
</plugins>
</build>
```

Run:

```
mvn package
java -jar target/myapp.jar
```

9. Building a WAR File

Create a Maven web project with structure:

```
src/main/webapp/
└─ WEB-INF/web.xml
```

Add:

```
<packaging>war</packaging>
```

Command:

```
mvn package
```

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.